

Primero sostener, después crecer

G. D. Monzón Flores
7690-20-20745 Universidad Mariano Gálvez
Seminario de Tecnología de Información
gmonzonf1@miumg.edu.gt

Resumen

El presente artículo analiza la importancia de elegir la arquitectura de una aplicación de acuerdo con sus necesidades reales, en lugar de adoptar modelos tecnológicos únicamente por su popularidad. El objetivo es identificar los aspectos que deben evaluarse antes de seleccionar una arquitectura y explicar cómo estas decisiones influyen en el crecimiento, mantenimiento y rendimiento de un sistema. Para ello, se realizó una revisión de documentación técnica relacionada con estilos arquitectónicos, sistemas monolíticos, microservicios y prácticas de diseño en la nube. El análisis permitió identificar que factores como el volumen de usuarios, la necesidad de disponibilidad, las integraciones externas, la seguridad, el presupuesto y la experiencia del equipo influyen directamente en la arquitectura más conveniente. Se concluye que no existe una arquitectura universalmente superior, ya que una solución compleja puede ser innecesaria para un proyecto pequeño, mientras que una solución demasiado simple puede limitar una aplicación que necesita crecer. La arquitectura correcta debe responder al contexto del proyecto y permitir su evolución sin convertir el mantenimiento en un problema.

Palabras clave: arquitectura de software, escalabilidad, monolito modular, microservicios, requisitos no funcionales.

Desarrollo del tema Cuando se habla de arquitectura de software, muchas veces se piensa en términos que pueden parecer demasiado técnicos, como microservicios, *serverless* o aplicaciones *cloud native*. Sin embargo, la arquitectura también representa una decisión de negocio. Define la manera en que una aplicación podrá evolucionar, responder a una mayor demanda, integrarse con otros servicios y mantenerse a lo largo del tiempo.

Una arquitectura mal elegida desde el inicio puede limitar el crecimiento de un sistema, dificultar y encarecer su mantenimiento o generar cuellos de botella conforme la aplicación escala. Por el contrario, una arquitectura alineada con los objetivos del proyecto permite evolucionar de forma ordenada, sin incorporar complejidad que no aporta valor. Microsoft (2025a) explica que un estilo arquitectónico representa una familia de soluciones que comparten determinadas características, por lo que no debe entenderse como una receta única que pueda aplicarse de la misma forma a todos los proyectos.

Antes de escribir una línea de código, la elección de una arquitectura debería partir de algunas preguntas estratégicas que permitan comprender las necesidades reales del negocio: ¿cuál es el volumen de usuarios esperado?, ¿el producto necesita crecer rápidamente?, ¿requiere alta disponibilidad?, ¿habrá integraciones frecuentes con terceros? y ¿qué tan crítica es la seguridad de la información? Estas preguntas permiten identificar los requisitos no funcionales del sistema, es decir, aquellos que no describen qué hará la aplicación, sino cómo debe comportarse mientras lo hace.

Por ejemplo, una plataforma utilizada internamente por veinte empleados no necesita la misma infraestructura que una tienda en línea que recibe miles de visitas durante una oferta. Ambas aplicaciones pueden incluir usuarios, reportes y una base de datos, pero sus necesidades de

disponibilidad, rendimiento y escalabilidad son completamente diferentes. Amazon Web Services (AWS, 2025) indica que las decisiones de arquitectura deben equilibrar aspectos como seguridad, confiabilidad, eficiencia del rendimiento, optimización de costos y excelencia operativa. Esto demuestra que elegir una arquitectura no consiste únicamente en buscar que el sistema funcione, sino en decidir qué cualidades son prioritarias para el tipo de aplicación que se construirá.

Una arquitectura monolítica modular puede ser una buena elección para proyectos que se encuentran en una etapa inicial o que poseen requerimientos estables. En este enfoque, las funciones principales se desarrollan dentro de una sola aplicación, pero se organizan en módulos separados según su responsabilidad. Por ejemplo, una aplicación administrativa puede dividirse internamente en módulos para usuarios, inventario, ventas y reportes, sin que cada uno de ellos tenga que ejecutarse como un servicio independiente. Esta estructura permite iniciar con menor complejidad, facilita las pruebas y reduce la cantidad de infraestructura que el equipo debe administrar.

Los microservicios representan una alternativa para sistemas que necesitan escalar partes específicas de la aplicación, integrar equipos diferentes o evolucionar funcionalidades de forma independiente. En lugar de contar con una única aplicación, el sistema se divide en servicios pequeños que se comunican entre sí mediante interfaces definidas. Una plataforma de comercio electrónico, por ejemplo, podría separar el catálogo de productos, los pagos, el inventario y las notificaciones en servicios distintos. De esta forma, si la consulta de productos recibe una demanda elevada, se puede aumentar la capacidad de ese servicio sin tener que escalar toda la plataforma.

No obstante, los microservicios no deben adoptarse únicamente porque son una tendencia. Cada servicio necesita despliegue, monitoreo, comunicación, manejo de errores y seguridad propios. Microsoft (2025b) indica que los microservicios deben ser autónomos y capaces de evolucionar de forma independiente; si las funcionalidades no pueden separarse realmente, dividir la aplicación solo agrega dependencias y complejidad. Un equipo pequeño que intenta administrar muchos servicios sin contar con experiencia en automatización, monitoreo o infraestructura podría terminar con un sistema más difícil de mantener que un monolito bien organizado.

Si el proyecto contempla un crecimiento acelerado, alto tráfico o picos constantes de solicitudes, la arquitectura debe prepararse para escalar de forma horizontal. Esto significa aumentar la cantidad de instancias disponibles en lugar de depender únicamente de un servidor más potente. En este contexto, los contenedores, el uso de servicios administrados y las aplicaciones preparadas para ejecutarse en la nube pueden alinearse mejor con los requerimientos. Sin embargo, escalar no significa necesariamente incorporar complejidad, sino contar con la capacidad de adaptación necesaria para responder al crecimiento real del sistema.

La seguridad también debe participar desde el momento en que se decide la arquitectura. Una aplicación que maneja pagos, datos médicos o información personal requiere controles diferentes a un sistema que únicamente publica información. La autenticación, los permisos de acceso, el cifrado y el registro de actividades no deberían agregarse al final como si fueran una función adicional. La arquitectura debe considerar desde el inicio qué información se protege, quién puede acceder a ella y qué ocurre si un componente falla o es atacado.

Por último, la experiencia del equipo y el presupuesto disponible son factores que no pueden ignorarse. Una arquitectura técnicamente adecuada puede fracasar si nadie cuenta con los conocimientos para desarrollarla y mantenerla. Si una organización no dispone de especialistas para administrar

una infraestructura distribuida, puede resultar más razonable comenzar con una solución simple, documentada y fácil de operar. La mejor arquitectura no es la que utiliza más tecnologías, sino la que resuelve el problema de manera clara, sostenible y proporcional a los recursos disponibles.

Observaciones y comentarios

Durante el análisis se observó que existe una tendencia a presentar los microservicios y las tecnologías en la nube como si fueran la solución obligatoria para cualquier sistema moderno. Sin embargo, esta idea puede llevar a tomar decisiones por moda y no por necesidad. Una aplicación pequeña, con pocos usuarios y procesos conocidos, puede funcionar correctamente con una arquitectura monolítica modular. Intentar dividirla desde el inicio en muchos servicios podría aumentar el tiempo de desarrollo, los costos de infraestructura y la dificultad para encontrar errores.

Como posible mejora, antes de seleccionar una arquitectura se podría elaborar una matriz de decisión que relacione los requerimientos del proyecto con factores como seguridad, escalabilidad, presupuesto, disponibilidad y experiencia del equipo. Esto ayudaría a justificar técnicamente la decisión y evitaría que la arquitectura dependa solamente de la preferencia personal de los desarrolladores.

Conclusiones

1. La arquitectura de una aplicación debe elegirse de acuerdo con las necesidades reales del negocio, los usuarios y los requisitos no funcionales del sistema.
2. Una arquitectura monolítica modular puede ser una solución adecuada para proyectos pequeños o con requerimientos estables, siempre que mantenga una organización interna clara.
3. Los microservicios pueden facilitar el escalamiento y la evolución independiente de ciertas funciones, pero también incrementan la complejidad técnica y operativa del proyecto.
4. La seguridad, disponibilidad, rendimiento, presupuesto y experiencia del equipo deben evaluarse antes de seleccionar una arquitectura, ya que todos influyen en la viabilidad del sistema.
5. La mejor arquitectura no es necesariamente la más moderna o compleja, sino aquella que permite resolver el problema actual y evolucionar de forma sostenible cuando sea necesario.

Referencias

Amazon Web Services. (2025). *Definitions*. AWS Well-Architected Framework. <https://docs.aws.amazon.com/wellarchitected/2025-02-25/framework/definitions.html>

Microsoft. (2025a). *Architecture styles*. Azure Architecture Center. <https://learn.microsoft.com/en-us/azure/architecture/guide/architecture-styles/>

Microsoft. (2025b). *Microservices architecture style*. Azure Architecture Center. <https://learn.microsoft.com/en-us/azure/architecture/guide/architecture-styles/microservices>

Repositorio

<https://github.com/DaniSaurioM/ForosAcademicos-Seminario>